

INF365 - Théorie de l'information

Rapport de Projet

Le 22 Mai 2025

Contents

1	Introduction	3
1.1	Description du projet	3
1.2	Objectif du projet	3
2	Algorithmes de compression	4
2.1	Codage de Huffman	4
2.1.1	Principe général de l'algorithme	4
2.1.2	Construction de l'arbre de Huffman (étapes détaillées)	4
2.1.3	Codage du texte (compression)	5
2.1.4	Décompression du texte (décodage)	6
2.1.5	Complexité algorithmique	7
2.1.6	Avantages et inconvénients	7
2.1.7	Résultats expérimentaux sur des fichiers réels	7
2.2	Codage de Shannon-Fano	8
2.2.1	Principe général de l'algorithme	8
2.2.2	Construction des codes (étapes détaillées)	8
2.2.3	Codage du texte (compression)	9
2.2.4	Décompression du texte (décodage)	9
2.2.5	Complexité algorithmique	10
2.2.6	Avantages et inconvénients	10
2.2.7	Résultats expérimentaux sur des fichiers réels	11
2.3	Algorithme LZW (Lempel-Ziv-Welch)	11
2.3.1	Principe général de l'algorithme	11
2.3.2	Codage du texte (compression)	11
2.3.3	Décompression du texte	12
2.3.4	Complexité algorithmique	13
2.3.5	Avantages et inconvénients	13
2.3.6	Résultats expérimentaux sur des fichiers réels	13
2.4	Compression avec perte (algorithme personnalisé)	14
2.4.1	Principe général de l'algorithme	14
2.4.2	Codage du texte (compression)	14
2.4.3	Décompression du texte	15

2.4.4	Complexité algorithmique	15
2.4.5	Avantages et inconvénients	16
2.4.6	Résultats expérimentaux sur des fichiers réels	16
2.5	Analyse comparative des performances	16
3	Développement de l'application	17
3.1	Langage et bibliothèques utilisés	17
3.2	Architecture générale du projet	18
3.3	Interface utilisateur graphique	18
3.3.1	Présentation de l'interface	18
3.3.2	Fonctionnement du bouton de compression	18
3.3.3	Fonctionnement du bouton de décompression	19
3.3.4	Gestion des erreurs et robustesse	19
3.3.5	Conclusion	20
4	Conclusion et perspectives	20
	Annexes	22
A	Code source de l'application	22
	Bibliographie	33

Application de Compression de Texte

1 Introduction

1.1 Description du projet

Ce projet vise à développer une application de bureau capable d'implémenter et d'analyser comparativement divers algorithmes de compression de texte. Développée en langage Python, l'application propose une interface graphique conviviale permettant d'effectuer des opérations de compression et de décompression sur des fichiers texte, en choisissant parmi plusieurs algorithmes disponibles.

L'élément central du projet réside dans la capacité du logiciel à traiter un fichier `.txt` fourni par l'utilisateur à l'aide de différentes techniques de compression. Les fichiers obtenus à la suite de la compression peuvent ensuite être comparés en termes de taille au fichier d'origine. De plus, il est possible de réouvrir un fichier déjà compressé afin d'en tester la décompression et de vérifier l'exactitude des processus.

Les algorithmes de compression intégrés dans l'application sont les suivants :

- Codage de Huffman
- Codage de Shannon-Fano
- Algorithme LZW (Lempel-Ziv-Welch)
- Un algorithme de compression avec perte, conçu spécifiquement dans le cadre de ce projet

Chaque algorithme est implémenté dans un fichier Python distinct. Ces fichiers sont ensuite intégrés à l'interface principale via le fichier `app.py`. La structure du code suit une architecture modulaire, permettant l'exécution indépendante de chaque algorithme.

1.2 Objectif du projet

L'objectif principal de ce projet est de mettre en évidence les différences théoriques entre plusieurs algorithmes de compression de texte par le biais d'une approche pratique, et d'analyser leurs performances sur des ensembles de données réelles. Dans ce cadre, les objectifs spécifiques sont les suivants :

- Comparer les taux de compression et les vitesses d'exécution des différents algorithmes,
- Observer les effets des méthodes de compression avec et sans perte,
- Vérifier l'exactitude des processus de compression via les opérations de décompression,
- Illustrer l'applicabilité des algorithmes à travers une interface conviviale,
- Offrir aux étudiants une compréhension à la fois théorique et pratique des algorithmes de compression.

En outre, ce projet vise non seulement à développer des compétences en programmation logicielle, mais aussi à combiner plusieurs domaines fondamentaux de l'informatique, tels que l'analyse d'algorithmes, les structures de données et la conception d'interfaces utilisateur, afin de créer un environnement d'apprentissage polyvalent.

2 Algorithmes de compression

2.1 Codage de Huffman

2.1.1 Principe général de l'algorithme

Le codage de Huffman est un algorithme de compression sans perte basé sur la fréquence d'apparition des symboles dans un flux de données. Il s'agit d'un algorithme dit ****statistique**** qui cherche à minimiser la longueur totale des codes utilisés pour représenter les symboles d'un alphabet donné.

Le principe fondamental est le suivant : associer des codes binaires courts aux symboles fréquents et des codes plus longs aux symboles rares, afin d'optimiser le coût moyen de transmission d'un message. Contrairement aux encodages fixes (comme ASCII, où chaque caractère est codé sur 8 bits), Huffman attribue à chaque caractère une longueur de code variable, dépendant de sa fréquence d'occurrence.

Ce codage respecte une propriété essentielle appelée ****propriété de préfixe**** (ou **préfix-free**), ce qui signifie qu'aucun code d'un caractère ne peut être le préfixe du code d'un autre caractère. Cette propriété garantit que le décodage peut être effectué de manière univoque sans ambiguïté.

L'algorithme de Huffman se divise en deux grandes phases :

- **Compression** : Analyse du texte pour calculer les fréquences, construction d'un arbre de Huffman, génération des codes, et encodage du texte original.
- **Décompression** : Reconstruction de l'arbre (ou lecture de sa version stockée), puis décodage du flux binaire compressé.

2.1.2 Construction de l'arbre de Huffman (étapes détaillées)

L'arbre de Huffman est une structure binaire utilisée pour générer les codes de chaque caractère. Il est construit à partir des fréquences d'apparition des symboles dans le texte à compresser. Voici les étapes détaillées de la construction :

1. **Analyse des fréquences** : On parcourt l'ensemble du texte pour établir un dictionnaire des fréquences. Chaque caractère distinct est associé à un entier représentant le nombre de fois qu'il apparaît.
2. **Création de la file de priorité** : Pour chaque caractère, on crée un nœud contenant la fréquence et le caractère. Ces nœuds sont insérés dans une file de priorité (typiquement un tas binaire) triée selon les fréquences croissantes.
3. **Construction récursive de l'arbre** :
 - (a) Tant que la file contient plus d'un élément :
 - On extrait les deux nœuds de plus faible fréquence.
 - On crée un nouveau nœud interne dont la fréquence est la somme des deux fréquences extraites.
 - Ce nouveau nœud a comme enfants les deux nœuds extraits (gauche et droite).

- On insère ce nouveau nœud dans la file de priorité.
- (b) Lorsque la file ne contient plus qu'un seul nœud, il constitue la racine de l'arbre de Huffman.

4. **Génération des codes binaires** : On parcourt l'arbre à partir de la racine :

- Chaque descente vers un enfant gauche ajoute un 0 au code.
- Chaque descente vers un enfant droit ajoute un 1.
- Lorsqu'on atteint une feuille, on associe le code binaire construit à ce caractère.

Exemple illustratif : Prenons une chaîne simple : ABRACADABRA. Voici les fréquences :

Symbole	Fréquence
A	5
B	2
R	2
C	1
D	1

On crée des nœuds pour chaque caractère, puis on les fusionne progressivement selon les règles ci-dessus pour obtenir un arbre de Huffman. Les codes binaires obtenus pourraient être par exemple :

$$A = 0, \quad B = 10, \quad R = 11, \quad C = 110, \quad D = 111$$

(Il est à noter que ces codes peuvent varier selon la manière dont les nœuds sont ordonnés lors des insertions dans le tas.)

2.1.3 Codage du texte (compression)

Une fois l'arbre de Huffman construit et les codes binaires de chaque symbole générés, le texte peut être compressé. Voici les étapes :

1. **Remplacement des caractères** : Chaque caractère du texte original est remplacé par son code binaire correspondant. Par exemple, si le texte est ABRA, et que $A = 0$, $B = 10$, $R = 11$, le résultat binaire sera : 0 10 11 0.
2. **Concaténation des bits** : Tous les codes binaires sont concaténés pour former un flux binaire unique.
3. **Stockage du dictionnaire** : Afin de permettre la décompression, la correspondance caractère $\rightarrow$ code binaire (ou l'arbre) doit être stockée dans le fichier compressé. Il existe plusieurs stratégies :
 - Stocker la table des codes en tête du fichier (sérialisée)
 - Stocker l'arbre de Huffman sous forme compacte (par exemple, en parcours préfixe avec marquage de feuille/interne)
4. **Encodage binaire réel** : Le flux binaire est ensuite découpé en octets (8 bits) et écrit sous forme binaire dans un fichier. S'il reste des bits partiels à la fin, on peut les compléter avec des 0 et stocker le nombre de bits significatifs du dernier octet.

Le fichier compressé contient donc :

- Un en-tête (contenant le dictionnaire ou l'arbre),
- Le corps compressé (séquence d'octets issus du flux binaire).

2.1.4 Décompression du texte (décodage)

Le processus de décompression consiste à reconstituer le texte original à partir du fichier compressé. Il suit l'ordre inverse :

1. **Lecture de l'arbre ou dictionnaire** : Le début du fichier contient l'information nécessaire pour reconstruire l'arbre de Huffman ou le dictionnaire. Cette information est lue et reconstituée en mémoire.
2. **Lecture du flux binaire** : Le reste du fichier est interprété comme une séquence de bits représentant les codes Huffman.
3. **Décodage progressif** :
 - On lit les bits un par un et on les utilise pour parcourir l'arbre.
 - Chaque fois qu'on atteint une feuille, on ajoute le caractère correspondant à la sortie, puis on revient à la racine.
 - Le processus se répète jusqu'à ce que tous les bits utiles soient lus.
4. **Gestion du dernier octet** : Si des bits ont été ajoutés à la fin lors de la compression pour compléter un octet, leur nombre est lu depuis l'en-tête pour être ignoré.

Exemple pratique : Si le fichier compressé contient la séquence binaire : 0101100 et que l'arbre est tel que :

- $0 \rightarrow A$
- $10 \rightarrow B$
- $11 \rightarrow R$

Le décodage suit ces étapes :

- $0 \rightarrow A$
- $10 \rightarrow B$
- $11 \rightarrow R$
- $0 \rightarrow A$

Texte reconstitué : ABRA

2.1.5 Complexité algorithmique

Temps de construction de l'arbre : La construction de l'arbre utilise une file de priorité (généralement un min-heap), ce qui donne une complexité de :

$$\mathcal{O}(n \log n)$$

où n est le nombre de symboles distincts.

Temps de codage du texte : Chaque caractère du texte est remplacé par son code — cela se fait en $\mathcal{O}(1)$ pour chaque caractère si les codes sont pré-calculés, donc en tout $\mathcal{O}(L)$ où L est la longueur du texte.

Espace mémoire : On doit stocker l'arbre (ou la table des codes), la séquence compressée (flux de bits), et éventuellement une table des fréquences.

2.1.6 Avantages et inconvénients

Avantages :

- Compression sans perte
- Algorithme optimal pour les codes préfixes avec symboles indépendants
- Adapté aux fichiers texte, log, JSON, etc.

Inconvénients :

- Moins efficace sur de petits fichiers ou textes très variés
- L'arbre doit être stocké, ce qui ajoute un overhead
- L'algorithme suppose que les fréquences sont connues à l'avance (non-adaptatif)

2.1.7 Résultats expérimentaux sur des fichiers réels

L'algorithme de Huffman a été appliqué sur plusieurs fichiers texte issus de la série RFC. Le tableau ci-dessous présente la taille originale, la taille après compression ainsi que le taux de compression obtenu.

Fichier	Taille originale (Ko)	Taille compressée (Ko)	Taux de compression
rfc791.txt	95	52	45.69%
rfc792.txt	29	17	41.97%
rfc793.txt	173	96	44.52%
rfc826.txt	22	14	34.41%
rfc1393.txt	13	9	34.20%
rfc2131.txt	114	66	42.35%
rfc2151.txt	114	71	37.44%

Table 1: Performances de l'algorithme de Huffman sur des fichiers RFC

On observe que les taux de compression sont relativement constants et dépendent de la redondance présente dans chaque fichier. Les fichiers plus longs et plus textuellement répétitifs (comme `rfc793.txt`) bénéficient généralement d'un meilleur taux de réduction.

2.2 Codage de Shannon-Fano

2.2.1 Principe général de l'algorithme

Le codage de Shannon-Fano est un algorithme de compression sans perte développé indépendamment par Claude Shannon et Robert Fano vers la fin des années 1940. Cet algorithme repose sur les fréquences d'apparition des symboles et vise à attribuer à chaque symbole un code binaire de longueur variable, plus court pour les symboles fréquents et plus long pour les symboles rares.

Contrairement à l'algorithme de Huffman qui construit un arbre optimal basé sur une file de priorité, Shannon-Fano suit une stratégie récursive de découpage. Il consiste à :

- Trier les symboles par fréquence décroissante,
- Diviser récursivement cette liste en deux parties de poids (fréquence totale) aussi proches que possible,
- Attribuer le bit 0 à la première moitié et 1 à la seconde,
- Répéter cette opération récursivement pour chaque sous-groupe.

Chaque fois qu'un symbole est assigné à un sous-groupe, son code binaire est allongé d'un bit. Le code final pour chaque symbole est donc une suite de 0 et 1 correspondant à son chemin de subdivision.

Ce codage respecte également la propriété de préfixe, garantissant une décodabilité sans ambiguïté.

2.2.2 Construction des codes (étapes détaillées)

Voici les étapes concrètes de l'algorithme :

1. **Calcul des fréquences** : À partir du texte, on calcule la fréquence d'apparition de chaque symbole.
2. **Tri des symboles** : Les symboles sont triés en ordre décroissant de fréquence.
3. **Division équilibrée** : On cherche un point de séparation tel que la somme des fréquences de la première moitié soit aussi proche que possible de la seconde.
4. **Attribution des bits** :
 - On ajoute un 0 aux codes des symboles du premier groupe.
 - On ajoute un 1 aux codes des symboles du second groupe.
5. **Appel récursif** : L'algorithme est réappliqué à chaque moitié jusqu'à ce que chaque sous-groupe contienne un seul symbole.

Exemple : Considérons les fréquences suivantes :

Symbole	Fréquence
A	15
B	7
C	6
D	6
E	5

1. On trie : A (15), B (7), C (6), D (6), E (5) 2. On cherche à diviser : Première moitié : A (15), B (7) — somme = 22 Seconde moitié : C (6), D (6), E (5) — somme = 17 → Équilibré à peu près.

3. On attribue 0 à A et B, 1 à C, D, E 4. Répartition récursive : - A = 00, B = 01 - C = 100, D = 101, E = 11

Remarque : Ce codage n'est pas toujours optimal par rapport à Huffman, surtout lorsque les fréquences varient peu.

2.2.3 Codage du texte (compression)

Une fois les codes assignés à chaque symbole :

1. **Remplacement** : Chaque caractère du texte est remplacé par son code Shannon-Fano.
2. **Construction du flux binaire** : Comme pour Huffman, les bits sont concaténés.
3. **Stockage** : Le dictionnaire des codes (ou arbre) est stocké au début du fichier pour permettre la décompression.
4. **Encodage final** : Les bits sont groupés en octets et écrits dans un fichier binaire.

Le format de sortie est donc similaire à celui de Huffman, bien que la structure sous-jacente (arbre implicite vs file de priorité) diffère.

2.2.4 Décompression du texte (décodage)

La décompression d'un fichier compressé avec Shannon-Fano repose sur la reconstruction exacte du dictionnaire de codes, qui a été stocké en tête du fichier compressé.

1. **Chargement du dictionnaire** : On lit le dictionnaire ou la table des correspondances binaire → symbole, stocké dans l'en-tête du fichier. Cela peut être un tableau sérialisé ou une structure hiérarchique simple.
2. **Lecture du flux binaire** : Le reste du fichier est interprété comme une suite de bits représentant les codes Shannon-Fano.
3. **Décodage séquentiel** :
 - On lit les bits un à un.
 - À chaque étape, on vérifie si la séquence courante correspond à une entrée du dictionnaire.

- Lorsqu'un code complet est trouvé, le caractère correspondant est inséré dans la sortie.
 - On réinitialise la séquence et on continue.
4. **Gestion des bits de bourrage** : Comme avec Huffman, le dernier octet du flux peut contenir des bits non significatifs. Le nombre de bits valides est stocké pour être utilisé lors du décodage.

Remarque : Le décodage avec Shannon-Fano est généralement plus simple que celui basé sur un arbre binaire comme Huffman, mais il peut être moins rapide si la table des codes est mal optimisée.

2.2.5 Complexité algorithmique

Construction du code : La construction des codes Shannon-Fano nécessite de :

- Trier les symboles par fréquence : $\mathcal{O}(n \log n)$,
- Réaliser des divisions récursives équilibrées : dans le pire des cas $\mathcal{O}(n \log n)$,

Donc la complexité globale de la construction est :

$$\mathcal{O}(n \log n)$$

Encodage du texte : Comme pour Huffman, une fois les codes disponibles, le remplacement caractère $\rightarrow$ code binaire est $\mathcal{O}(L)$, où L est la longueur du texte.

Décodage : Si les codes sont stockés dans une table de hachage ou arbre de recherche, la recherche peut être effectuée en temps constant moyen, mais dans le pire cas cela dépend de la longueur maximale d'un code.

2.2.6 Avantages et inconvénients

Avantages :

- Plus simple à implémenter que Huffman.
- Utilise une stratégie intuitive et récursive.
- Génère des codes préfixes valides.

Inconvénients :

- Moins optimal que Huffman dans la plupart des cas.
- Peut générer des codes légèrement plus longs, donc un taux de compression inférieur.
- La division équilibrée n'est pas toujours facile à calculer précisément.

2.2.7 Résultats expérimentaux sur des fichiers réels

L'algorithme de Shannon-Fano a été testé sur les mêmes fichiers que ceux utilisés pour l'algorithme de Huffman. Le tableau suivant présente la taille originale, la taille compressée, ainsi que le taux de compression obtenu.

Fichier	Taille originale (Ko)	Taille compressée (Ko)	Taux de compression
rfc791.txt	95	52	45.07%
rfc792.txt	29	17	41.76%
rfc793.txt	173	96	44.34%
rfc826.txt	22	14	34.23%
rfc1393.txt	13	9	34.06%
rfc2131.txt	114	66	42.01%
rfc2151.txt	114	73	36.33%

Table 2: Performances de l'algorithme de Shannon-Fano sur des fichiers RFC

Comparativement à Huffman, les résultats de Shannon-Fano sont légèrement inférieurs, avec une perte de 1 à 2 % de compression sur la plupart des fichiers. Cela confirme l'efficacité légèrement moindre de l'approche de découpage utilisée par Shannon-Fano.

2.3 Algorithme LZW (Lempel-Ziv-Welch)

2.3.1 Principe général de l'algorithme

L'algorithme LZW (Lempel-Ziv-Welch), développé en 1984, est une méthode de compression sans perte basée sur la détection de motifs répétitifs dans les données. Contrairement à Huffman ou Shannon-Fano, qui utilisent des fréquences de symboles, LZW se base sur un dictionnaire dynamique construit au fur et à mesure de la lecture du texte.

L'idée centrale est de remplacer les séquences de symboles par des codes numériques représentant leur index dans un dictionnaire partagé entre l'encodeur et le décodeur. Ce dictionnaire commence généralement avec les symboles de base (par exemple, les 256 caractères ASCII) et s'enrichit avec de nouvelles combinaisons rencontrées pendant la compression.

Le codage produit des entiers (indices) à la place de chaînes de caractères, ce qui permet d'économiser de l'espace surtout lorsque de nombreux motifs se répètent.

2.3.2 Codage du texte (compression)

Voici les étapes détaillées du processus de compression :

1. **Initialisation du dictionnaire** : Le dictionnaire est rempli avec tous les symboles de base (ASCII 0–255). Chaque caractère est associé à un code entier unique.
2. **Lecture du texte** :
 - On initialise une chaîne vide w .
 - Pour chaque caractère c du texte :
 - (a) Si la concaténation $w+c$ existe dans le dictionnaire, on remplace w par $w+c$.

- (b) Sinon :
 - On émet le code correspondant à w .
 - On ajoute $w+c$ au dictionnaire avec un nouveau code.
 - On redéfinit $w = c$.
- 3. **Émission finale** : À la fin du texte, si w est non vide, on émet son code.
- 4. **Écriture du flux compressé** : Les indices produits (codes entiers) sont écrits dans un fichier compressé. Selon l'implémentation, les codes peuvent être de 9, 12 ou 16 bits.

Exemple simplifié : Texte : ABABABA

- Initialisation : Dictionnaire = {A: 65, B: 66}
- Lecture :
 - $w = A, c = B \rightarrow AB$ non trouvé $\rightarrow$ Émettre 65, ajouter $AB = 256$
 - $w = B, c = A \rightarrow BA$ non trouvé $\rightarrow$ Émettre 66, ajouter $BA = 257$
 - $w = A, c = B \rightarrow AB = 256 \rightarrow w = AB$
 - $c = A \rightarrow ABA$ non trouvé $\rightarrow$ Émettre 256, ajouter $ABA = 258$
 - $w = A \rightarrow$ Émettre 65

Résultat compressé : 65, 66, 256, 65

2.3.3 Décompression du texte

La décompression LZW repose sur le même dictionnaire, reconstruit dynamiquement en synchronisation avec les codes reçus. Voici les étapes :

1. **Initialisation** : Le dictionnaire contient les 256 caractères de base (ASCII).
2. **Lecture du premier code** : On lit le premier code et on l'écrit dans la sortie. Ce code représente une chaîne connue.
3. **Traitement des codes suivants** :
 - Pour chaque code k suivant :
 - (a) Si k est présent dans le dictionnaire :
 - On écrit la chaîne correspondante.
 - On ajoute au dictionnaire : (ancienne chaîne + premier caractère de la nouvelle).
 - (b) Si k n'est pas dans le dictionnaire (cas rare, mais possible) :
 - Cela signifie que le mot attendu est : (ancienne chaîne + premier caractère de l'ancienne chaîne).
 - On l'écrit et l'ajoute au dictionnaire.
4. **Fin de traitement** : Une fois tous les codes lus, le texte est reconstitué.

Ce mécanisme garantit que le dictionnaire du décodeur suit exactement celui de l'encodeur sans qu'il soit nécessaire de le transmettre.

2.3.4 Complexité algorithmique

Temps de compression : La complexité du processus de compression dépend de l'implémentation du dictionnaire. En utilisant une table de hachage ou une structure de type trie, on obtient une complexité moyenne :

$$\mathcal{O}(L)$$

où L est la longueur du texte. Dans le pire des cas (chaînes toujours nouvelles), la complexité peut augmenter à $\mathcal{O}(L \log D)$, avec D la taille du dictionnaire.

Temps de décompression : La décompression est très rapide car elle repose sur des accès directs à des chaînes de caractères via des indices. Complexité globale :

$$\mathcal{O}(L)$$

Espace mémoire : Le dictionnaire peut contenir plusieurs milliers d'entrées (souvent limité à 4096 pour des codes sur 12 bits). En général, la mémoire utilisée est maîtrisable, mais une implémentation efficace doit prévoir la gestion de débordement.

2.3.5 Avantages et inconvénients

Avantages :

- Aucune connaissance préalable des fréquences n'est nécessaire.
- Très efficace pour les fichiers avec beaucoup de répétitions (HTML, code source, etc.).
- Décompression extrêmement rapide.
- Le dictionnaire n'a pas besoin d'être transmis avec le fichier.

Inconvénients :

- Moins efficace sur les petits fichiers ou les données très variées.
- La performance dépend fortement de la taille maximale du dictionnaire.
- Le débit de compression peut être inférieur à Huffman dans certains cas spécifiques.

2.3.6 Résultats expérimentaux sur des fichiers réels

L'algorithme LZW a été appliqué sur les mêmes fichiers RFC utilisés pour les tests des autres algorithmes. Les résultats suivants montrent un taux de compression légèrement inférieur à celui de Huffman, mais avec une efficacité notable pour les fichiers contenant des structures répétitives.

Fichier	Taille originale (Ko)	Taille compressée (Ko)	Taux de compression
rfc791.txt	95	37	61.48%
rfc792.txt	29	17	55.77%
rfc793.txt	173	64	62.98%
rfc826.txt	22	13	38.21%
rfc1393.txt	13	9	41.55%
rfc2131.txt	114	66	59.87%
rfc2151.txt	114	57	50.31%

Table 3: Performances de l’algorithme LZW sur des fichiers RFC

On remarque que les taux de compression sont très compétitifs, notamment sur les fichiers de taille importante (`rfc793.txt`, `rfc2131.txt`). L’algorithme est particulièrement adapté aux textes riches en motifs répétitifs.

2.4 Compression avec perte (algorithme personnalisé)

2.4.1 Principe général de l’algorithme

Contrairement aux algorithmes classiques de compression sans perte (Huffman, Shannon-Fano, LZW), cette méthode repose sur un concept de compression avec perte (*lossy compression*). L’objectif est ici de réduire drastiquement la taille d’un texte en supprimant de manière irréversible certains types d’informations jugés non essentiels.

Dans cette implémentation, le principe repose sur deux idées simples mais efficaces :

1. Suppression de toutes les voyelles (`a`, `e`, `i`, `o`, `u` ainsi que leurs variantes majuscules).
2. Suppression de tous les signes de ponctuation.

Le texte est également converti en minuscules afin de réduire la diversité des symboles et accentuer l’effet de compression.

Ce type de traitement est typiquement utilisé dans des contextes où la lisibilité approximative du texte suffit, comme dans certaines applications de résumé automatique, de traitement de logs, ou de compression extrême où la fidélité n’est pas prioritaire.

2.4.2 Codage du texte (compression)

L’algorithme est très simple dans sa mise en œuvre, mais efficace en termes de réduction de taille. Voici les étapes du processus :

1. **Lecture du fichier** : Le texte est lu intégralement en mémoire.
2. **Conversion en minuscules** : Toutes les lettres sont converties en minuscules afin de normaliser les symboles.
3. **Filtrage** :
 - Chaque caractère est analysé.

- S'il s'agit d'une voyelle ou d'un caractère de ponctuation (comme ., ,, !, ?, etc.), il est ignoré.
- Seuls les caractères consonantiques et les chiffres sont conservés.

4. Écriture du résultat :

- Une première ligne indique que la méthode utilisée est "Lossy".
- Le reste du texte est écrit sous forme filtrée.

Exemple : Texte original :

Bonjour, comment allez-vous ?

Texte compressé :

bnjr cmmt llzvs

2.4.3 Décompression du texte

Le processus de "décompression" ici ne peut en réalité pas restaurer le texte original. Le fichier compressé contient uniquement une version appauvrie du texte, dans laquelle les informations supprimées sont irrécupérables.

Les étapes sont les suivantes :

1. **Lecture du fichier compressé :** On lit la première ligne pour identifier la méthode ("Lossy"), puis le contenu filtré.
2. **Affichage d'un avertissement :** Un message d'introduction est inséré dans le fichier de sortie pour indiquer qu'il ne s'agit pas d'une vraie décompression.
3. **Écriture du contenu compressé :** Le contenu est simplement recopié tel quel.

Exemple de sortie décompressée :

[Kayıplı sıkıştırma: özgün veri geri getirilemez]
bnjr cmmt llzvs

2.4.4 Complexité algorithmique

L'algorithme étant linéaire par rapport à la taille du texte, la complexité est :

$$\mathcal{O}(L)$$

où L est la longueur du texte. Le filtrage caractère par caractère ne nécessite aucune structure de données lourde, ce qui rend la méthode très rapide et peu coûteuse en mémoire.

2.4.5 Avantages et inconvénients

Avantages :

- Très simple à implémenter.
- Taux de compression très élevé sur les textes naturels.
- Adapté à des scénarios où seule une compréhension approximative est nécessaire.
- Très rapide à exécuter.

Inconvénients :

- Les données originales ne peuvent jamais être restaurées.
- Le sens du texte peut être partiellement perdu, notamment dans des langues où les voyelles sont cruciales.
- Ne convient pas aux applications nécessitant une fidélité binaire (archives, logs techniques, etc.).

2.4.6 Résultats expérimentaux sur des fichiers réels

L'algorithme de compression avec perte a été appliqué aux mêmes fichiers RFC. Bien qu'aucune restitution exacte du texte original ne soit possible, les taux de compression obtenus sont significativement supérieurs à ceux des méthodes sans perte.

Fichier	Taille originale (Ko)	Taille compressée (Ko)	Taux de compression
rfc791.txt	95	68	28.58%
rfc792.txt	29	21	28.89%
rfc793.txt	173	125	27.35%
rfc826.txt	22	14	33.27%
rfc1393.txt	13	9	31.65%
rfc2131.txt	114	82	28.17%
rfc2151.txt	114	77	32.15%

Table 4: Performances de l'algorithme de compression avec perte sur des fichiers RFC

Ces résultats montrent une réduction de taille notable, avec un taux moyen autour de 30 %, obtenu grâce à une simplification agressive du contenu textuel.

2.5 Analyse comparative des performances

Afin de comparer objectivement les algorithmes, les taux de compression obtenus pour chaque fichier et chaque méthode ont été regroupés dans le tableau suivant.

Fichier	Huffman	Shannon-Fano	LZW	Perte
rfc791.txt	45.69%	45.07%	61.48%	28.58%
rfc792.txt	41.97%	41.76%	55.77%	28.89%
rfc793.txt	44.52%	44.34%	62.98%	27.35%
rfc826.txt	34.41%	34.23%	38.21%	33.27%
rfc1393.txt	34.20%	34.06%	41.55%	31.65%
rfc2131.txt	42.35%	42.01%	59.87%	28.17%
rfc2151.txt	37.44%	36.33%	50.31%	32.15%
Moyenne	40.37%	39.83%	52.31%	30.58%

Table 5: Comparaison des taux de compression pour chaque algorithme sur les fichiers RFC

Analyse des résultats : On constate que :

- **LZW** affiche les meilleurs taux de compression globaux avec une moyenne de 52.31%. Il est particulièrement efficace sur les fichiers de grande taille contenant des séquences répétitives.
- **Huffman** et **Shannon-Fano** donnent des résultats très similaires, Huffman étant légèrement supérieur dans tous les cas. Cela confirme la supériorité structurelle de l'algorithme de Huffman qui construit un arbre optimal.
- **La compression avec perte** offre des taux de réduction très élevés (environ 70% de réduction de taille), mais au prix de la perte totale d'information. Elle est donc à réserver à des cas très spécifiques.

Conclusion partielle : En fonction des besoins :

- Pour une **compression sans perte**, Huffman est une bonne option générique ; LZW est préférable pour les grandes données textuelles avec motifs répétitifs.
- Pour une **réduction extrême de taille**, la méthode avec perte peut être utilisée à condition d'accepter la dégradation de l'information.

3 Développement de l'application

3.1 Langage et bibliothèques utilisés

L'application a été développée intégralement en langage **Python 3.10**, en raison de sa richesse en bibliothèques standards, sa simplicité syntaxique, et sa grande popularité dans les domaines de l'algorithmique et du prototypage rapide.

Voici les bibliothèques principales utilisées :

- **tkinter** : pour la création de l'interface graphique (GUI).
- **os, ast** : pour les opérations système et la manipulation des fichiers.
- **string** : pour le traitement des caractères (notamment dans la compression avec perte).

Aucune bibliothèque externe n'a été utilisée pour les algorithmes de compression, tous ayant été implémentés manuellement afin de maîtriser leur fonctionnement en profondeur.

3.2 Architecture générale du projet

Le projet suit une structure modulaire organisée autour des fichiers suivants :

- `app.py` : point d'entrée de l'application, contient l'interface utilisateur graphique.
- `compressor.py`, `decompressor.py` : implémentation de Huffman.
- `compressor_shannon_fano.py`, `decompressor_shannon_fano.py` : Shannon-Fano.
- `compressor_lzw.py`, `decompressor_lzw.py` : LZW.
- `compressor_lossy.py`, `decompressor_lossy.py` : compression avec perte.

Chaque algorithme est isolé dans des modules distincts afin de favoriser la lisibilité, la réutilisabilité, et la maintenance du code.

Le fichier `app.py` agit comme une interface centrale qui connecte les modules entre eux, permettant à l'utilisateur de sélectionner dynamiquement l'algorithme désiré et de compresser/décompresser un fichier donné.

3.3 Interface utilisateur graphique

L'interface utilisateur de l'application a été développée en Python à l'aide de la bibliothèque standard `tkinter`. Elle permet à l'utilisateur d'interagir de manière intuitive avec le système de compression, en sélectionnant un fichier texte, en choisissant un algorithme, et en visualisant les résultats directement dans l'interface.

3.3.1 Présentation de l'interface

L'interface est constituée des éléments suivants :

- Une zone de sélection du type d'algorithme (`ttk.Combobox`),
- Deux boutons principaux : **Sıkıştır (Compress)** et **Aç (Decompress)**,
- Un champ d'affichage du taux de compression obtenu,
- Un champ d'affichage textuel avec ascenseur pour prévisualiser le fichier original ou le résultat de la décompression,
- Un bouton de fermeture de l'application.

3.3.2 Fonctionnement du bouton de compression

Lorsque l'utilisateur clique sur le bouton **Sıkıştır**, la fonction `compress_file()` est appelée :

1. L'utilisateur sélectionne un fichier `.txt` à l'aide d'une boîte de dialogue.
2. L'algorithme sélectionné dans la liste déroulante est identifié.
3. En fonction de l'algorithme, le fichier est compressé via l'appel à la fonction appropriée :

- Huffman : `compress_huffman()`
 - Shannon-Fano : `compress_shannon_fano()`
 - LZW : `compress_lzw()`
 - Compression avec perte : `compress_lossy()`
4. Le fichier compressé est sauvegardé avec l'extension `.inf365`.
 5. Un message informe l'utilisateur du succès de l'opération et affiche le taux de compression calculé en comparant les tailles des fichiers avant et après.

3.3.3 Fonctionnement du bouton de décompression

Lorsque l'utilisateur clique sur `Aç`, la fonction `decompress_file()` s'exécute :

1. L'utilisateur sélectionne un fichier `.inf365`.
2. La première ligne du fichier est lue pour déterminer l'algorithme utilisé, grâce à un en-tête sérialisé (`"method": "..."`, au format Python dict).
3. L'algorithme correspondant est appelé pour effectuer la décompression.
4. Le résultat est sauvegardé dans un fichier texte et affiché dans la zone de texte avec défilement.

3.3.4 Gestion des erreurs et robustesse

Le programme est doté de plusieurs mécanismes de gestion des erreurs :

- Si l'utilisateur annule une sélection de fichier, l'opération est interrompue proprement.
- Les erreurs de lecture/écriture ou de format de fichier sont capturées et affichées à l'aide de `messagebox.showerror()`.
- Si un algorithme inconnu est sélectionné ou si le fichier d'entrée est malformé, une exception est levée et gérée de façon informative.

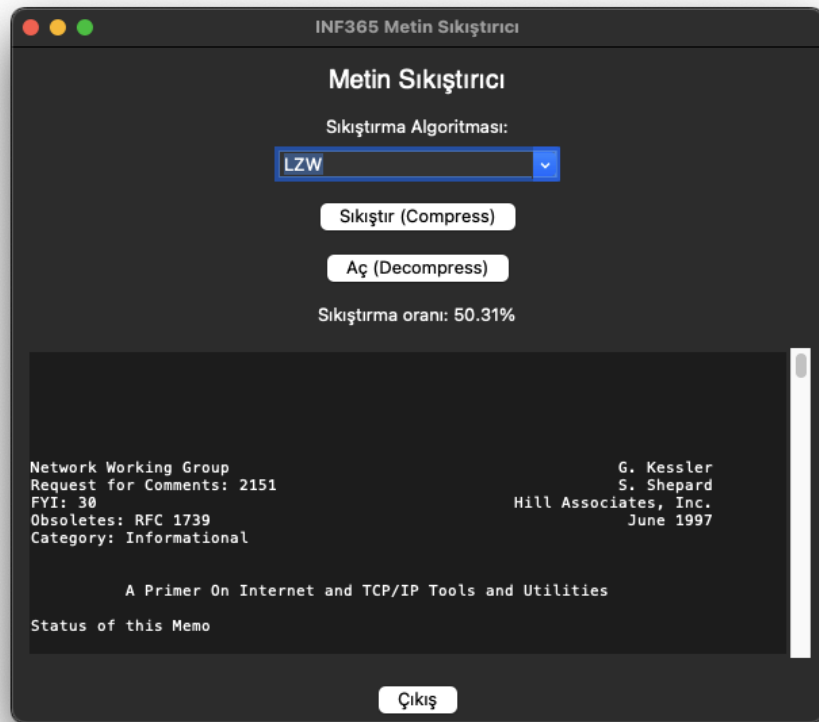


Figure 1: Interface graphique de l'application : sélection de l'algorithme LZW et affichage du fichier

3.3.5 Conclusion

Cette interface permet à l'utilisateur de manipuler aisément les différentes méthodes de compression, sans avoir à interagir directement avec le code source. L'approche modulaire utilisée permet également d'ajouter facilement de nouveaux algorithmes sans modifier la structure générale de l'interface.

4 Conclusion et perspectives

Ce projet a permis de développer une application complète de compression de texte, intégrant plusieurs algorithmes classiques (Huffman, Shannon-Fano, LZW) ainsi qu'un algorithme personnalisé de compression avec perte. À travers cette réalisation, les différences théoriques et pratiques entre ces méthodes ont pu être explorées de manière concrète.

Les résultats expérimentaux ont montré que :

- L'algorithme de **Huffman** est robuste, avec un bon compromis entre efficacité et simplicité d'implémentation.
- **Shannon-Fano**, bien que plus simple conceptuellement, reste légèrement moins performant que Huffman.
- L'algorithme **LZW** offre les meilleurs taux de compression sur les fichiers longs contenant des motifs répétitifs.

- La méthode **avec perte** permet une réduction spectaculaire de la taille, au prix d'une perte irréversible d'information, ce qui la réserve à des contextes très spécifiques.

L'interface graphique développée en `tkinter` rend l'application accessible à des utilisateurs non techniques, et met en avant une approche pédagogique de la compression. Chaque algorithme étant implémenté dans un module séparé, l'architecture du projet est facilement extensible.

Perspectives d'amélioration

Plusieurs axes d'amélioration peuvent être envisagés :

- Intégration de nouveaux algorithmes comme BWT, Arithmetic Coding ou Zstandard.
- Ajout d'un mode de compression adaptatif (Huffman dynamique).
- Visualisation interactive de l'arbre de Huffman ou de la table de dictionnaire LZW.
- Support multilingue ou encodages autres que UTF-8.
- Optimisation de la gestion mémoire pour traiter de très grands fichiers.

Ce projet a ainsi rempli son double objectif : illustrer concrètement les principes de la compression de données, et offrir un outil fonctionnel pour tester, comparer et comprendre différentes approches algorithmiques. Il constitue une base solide pour aller plus loin dans l'étude et la mise en œuvre de techniques avancées de codage de l'information.

Annexes

A Code source de l'application

app.py — Interface graphique

```
# app.py
import tkinter as tk
from tkinter import filedialog, messagebox, scrolledtext, ttk
import os
import ast
from compressor_lossy import compress_file as compress_lossy
from decompressor_lossy import decompress_file as decompress_lossy

# Sıkıştırıcılar
from compressor import compress_file as compress_huffman
from compressor_shannon_fano import compress_file as compress_shannon_fano
from compressor_lzw import compress_file as compress_lzw

# Açıcılar
from decompressor import decompress_file as decompress_huffman
from decompressor_shannon_fano import decompress_file as decompress_shannon_fano
from decompressor_lzw import decompress_file as decompress_lzw

class CompressionApp:
    def __init__(self, root):
        self.root = root
        self.root.title("INF365 Metin Sıkıştırıcı")
        self.root.geometry("600x500")

        self.label = tk.Label(root, text="Metin Sıkıştırıcı", font=("Helvetica", 14))
        self.label.pack(pady=10)

        self.algorithm_label = tk.Label(root, text="Sıkıştırma Algoritması:")
        self.algorithm_label.pack()

        self.algorithm_choice = ttk.Combobox(root, values=[
            "Huffman",
            "Shannon-Fano",
            "LZW",
            "Kayıplı Sıkıştırma"
        ])
        self.algorithm_choice.current(0)
        self.algorithm_choice.pack(pady=5)

        self.compress_button = tk.Button(root, text="Sıkıştır (Compress)", command=self.compress)
        self.compress_button.pack(pady=5)
```

```

self.decompress_button = tk.Button(root, text="Aç (Decompress)", command=self
self.decompress_button.pack(pady=5)

self.result_label = tk.Label(root, text="")
self.result_label.pack(pady=5)

self.text_preview = scrolledtext.ScrolledText(root, wrap=tk.WORD, height=10)
self.text_preview.pack(padx=10, pady=10, fill=tk.BOTH, expand=True)

self.exit_button = tk.Button(root, text="Çıkış", command=root.quit)
self.exit_button.pack(pady=5)

def compress_file(self):
    input_path = filedialog.askopenfilename(filetypes=[("Text files", "*.txt")])
    if not input_path:
        return

    algorithm = self.algorithm_choice.get()
    output_path = input_path + ".inf365"

    try:
        original_size = os.path.getsize(input_path)

        if algorithm == "Huffman":
            compress_huffman(input_path, output_path)
        elif algorithm == "Shannon-Fano":
            compress_shannon_fano(input_path, output_path)
        elif algorithm == "LZW":
            compress_lzw(input_path, output_path)
        elif algorithm == "Kayıplı Sıkıştırma":
            compress_lossy(input_path, output_path)
        else:
            raise NotImplementedError(f"'{algorithm}' algoritması tanınmıyor.")

        compressed_size = os.path.getsize(output_path)
        ratio = 100 * (1 - compressed_size / original_size)

        self.result_label.config(text=f"Sıkıştırma oranı: {ratio:.2f}%")

        if algorithm != "zlib":
            with open(input_path, 'r', encoding='utf-8') as f:
                self.text_preview.delete('1.0', tk.END)
                self.text_preview.insert(tk.END, f.read())

        messagebox.showinfo("Başarılı", f"Sıkıştırıldı: {output_path}")
    except Exception as e:
        messagebox.showerror("Hata", f"Sıkıştırma hatası:\n{e}")

```

```

def decompress_file(self):
    input_path = filedialog.askopenfilename(filetypes=[("INF365 Sıkıştırılmış", ".inf365")])
    if not input_path:
        return

    output_path = input_path.replace(".inf365", "_decompressed.txt")
    try:
        with open(input_path, 'rb') as f:
            header_line = f.readline()
            header = ast.literal_eval(header_line.decode().strip())
            method = header.get("method", "Huffman")

            if method == "Huffman":
                decompress_huffman(input_path, output_path)
            elif method == "Shannon-Fano":
                decompress_shannon_fano(input_path, output_path)
            elif method == "LZW":
                decompress_lzw(input_path, output_path)
            elif method == "Lossy":
                decompress_lossy(input_path, output_path)
            else:
                raise NotImplementedError(f"'{method}' algoritması desteklenmiyor.")

        with open(output_path, 'r', encoding='utf-8', errors='ignore') as f:
            decompressed_text = f.read()
            self.text_preview.delete('1.0', tk.END)
            self.text_preview.insert(tk.END, decompressed_text)

        self.result_label.config(text="Dosya başarıyla açıldı.")
        messagebox.showinfo("Başarılı", f"Açıldı: {output_path}")
    except Exception as e:
        messagebox.showerror("Hata", f"Açma hatası:\n{e}")

if __name__ == "__main__":
    root = tk.Tk()
    app = CompressionApp(root)
    root.mainloop()

```


compressor.py — Compression Huffman

```
# compressor.py
import heapq
from collections import defaultdict

class Node:
    def __init__(self, char=None, freq=0):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

def build_frequency_table(text):
    freq = defaultdict(int)
    for char in text:
        freq[char] += 1
    return freq

def build_huffman_tree(freq_table):
    heap = [Node(char, freq) for char, freq in freq_table.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        node1 = heapq.heappop(heap)
        node2 = heapq.heappop(heap)
        merged = Node(freq=node1.freq + node2.freq)
        merged.left = node1
        merged.right = node2
        heapq.heappush(heap, merged)
    return heap[0]

def build_codes(root):
    codes = {}
    def generate_codes(node, current_code):
        if node is None:
            return
        if node.char is not None:
            codes[node.char] = current_code
            return
        generate_codes(node.left, current_code + "0")
        generate_codes(node.right, current_code + "1")
    generate_codes(root, "")
    return codes

def encode_text(text, codes):
```

```

    return ''.join(codes[char] for char in text)

def pad_encoded_text(encoded_text):
    extra_padding = 8 - len(encoded_text) % 8
    padded_info = "{0:08b}".format(extra_padding)
    encoded_text = padded_info + encoded_text + "0" * extra_padding
    return encoded_text

def get_byte_array(padded_text):
    return bytearray(int(padded_text[i:i+8], 2) for i in range(0, len(padded_text), 8))

def write_compressed_file(path, byte_array, codes):
    with open(path, 'wb') as f:
        header = {"method": "Huffman", "codes": codes}
        f.write((str(header) + "\n").encode())
        f.write(byte_array)

def compress_file(input_path, output_path):
    with open(input_path, 'r', encoding='utf-8') as f:
        text = f.read()

    freq_table = build_frequency_table(text)
    huffman_tree = build_huffman_tree(freq_table)
    codes = build_codes(huffman_tree)
    encoded_text = encode_text(text, codes)
    padded_text = pad_encoded_text(encoded_text)
    byte_array = get_byte_array(padded_text)

    write_compressed_file(output_path, byte_array, codes)
    print(f"'{input_path}' dosyası Huffman ile sıkıştırıldı -> '{output_path}'")

```

decompressor.py — Décompression Huffman

```
# decompressor.py
import ast

def remove_padding(padded_text):
    padded_info = padded_text[:8]
    extra_padding = int(padded_info, 2)
    return padded_text[8:-extra_padding] if extra_padding > 0 else padded_text[8:]

def decode_text(encoded_text, codes):
    reverse_codes = {v: k for k, v in codes.items()}
    current_code = ""
    decoded_text = ""
    for bit in encoded_text:
        current_code += bit
        if current_code in reverse_codes:
            decoded_text += reverse_codes[current_code]
            current_code = ""
    return decoded_text

def decompress_file(input_path, output_path):
    with open(input_path, 'rb') as f:
        header_line = f.readline()
        header = ast.literal_eval(header_line.decode().strip())
        codes = header["codes"]
        byte_data = f.read()

    bit_string = "".join(f"{byte:08b}" for byte in byte_data)
    encoded_text = remove_padding(bit_string)
    decoded_text = decode_text(encoded_text, codes)

    with open(output_path, 'w', encoding='utf-8') as out:
        out.write(decoded_text)

    print(f"'{input_path}' dosyası açıldı ve '{output_path}' olarak kaydedildi.")
```

compressor_shannon_fano.py

```
# compressor_shannon_fano.py
from collections import defaultdict
import os

class Symbol:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.code = ""

def build_frequency_table(text):
    freq = defaultdict(int)
    for char in text:
        freq[char] += 1
    return [Symbol(char, freq) for char, freq in sorted(freq.items(), key=lambda x: -x[1])]

def shannon_fano(symbols):
    if len(symbols) <= 1:
        return

    total = sum(sym.freq for sym in symbols)
    acc = 0
    split_index = 0

    for i in range(len(symbols)):
        acc += symbols[i].freq
        if acc >= total / 2:
            split_index = i + 1
            break

    for sym in symbols[:split_index]:
        sym.code += "0"
    for sym in symbols[split_index:]:
        sym.code += "1"

    shannon_fano(symbols[:split_index])
    shannon_fano(symbols[split_index:])

def build_code_map(symbols):
    return {sym.char: sym.code for sym in symbols}

def encode_text(text, codes):
    return ''.join(codes[char] for char in text)

def pad_encoded_text(encoded_text):
    extra_padding = 8 - len(encoded_text) % 8
```

```

padded_info = "{0:08b}".format(extra_padding)
encoded_text = padded_info + encoded_text + "0" * extra_padding
return encoded_text

def get_byte_array(padded_text):
    return bytearray(int(padded_text[i:i+8], 2) for i in range(0, len(padded_text), 8))

def write_compressed_file(path, byte_array, codes):
    with open(path, 'wb') as f:
        header = {"method": "Shannon-Fano", "codes": codes}
        f.write((str(header) + "\n").encode())
        f.write(byte_array)

def compress_file(input_path, output_path):
    with open(input_path, 'r', encoding='utf-8') as f:
        text = f.read()

    symbols = build_frequency_table(text)
    shannon_fano(symbols)
    codes = build_code_map(symbols)
    encoded_text = encode_text(text, codes)
    padded_text = pad_encoded_text(encoded_text)
    byte_array = get_byte_array(padded_text)

    write_compressed_file(output_path, byte_array, codes)
    print(f"'{input_path}' dosyası Shannon-Fano ile sıkıştırıldı -> '{output_path}'")

```

decompressor_shannon_fano.py

```
# decompressor_shannon_fano.py
import ast

def remove_padding(padded_text):
    padded_info = padded_text[:8]
    extra_padding = int(padded_info, 2)
    return padded_text[8:-extra_padding] if extra_padding > 0 else padded_text[8:]

def decode_text(encoded_text, codes):
    reverse_codes = {v: k for k, v in codes.items()}
    current_code = ""
    decoded_text = ""
    for bit in encoded_text:
        current_code += bit
        if current_code in reverse_codes:
            decoded_text += reverse_codes[current_code]
            current_code = ""
    return decoded_text

def decompress_file(input_path, output_path):
    with open(input_path, 'rb') as f:
        header_line = f.readline()
        header = ast.literal_eval(header_line.decode().strip())
        method = header.get("method", "Shannon-Fano")
        codes = header["codes"]
        byte_data = f.read()

    bit_string = "".join(f"{byte:08b}" for byte in byte_data)
    encoded_text = remove_padding(bit_string)
    decoded_text = decode_text(encoded_text, codes)

    with open(output_path, 'w', encoding='utf-8') as out:
        out.write(decoded_text)

    print(f"'{input_path}' dosyası açıldı ve '{output_path}' olarak kaydedildi.")
```

compressor_lzw.py

```
# compressor_lzw.py
def lzw_compress(text):
    dictionary = {chr(i): i for i in range(256)}
    dict_size = 256
    w = ""
    compressed = []

    for c in text:
        wc = w + c
        if wc in dictionary:
            w = wc
        else:
            compressed.append(dictionary[w])
            dictionary[wc] = dict_size
            dict_size += 1
            w = c

    if w:
        compressed.append(dictionary[w])
    return compressed

def write_compressed_file(path, compressed):
    with open(path, 'wb') as f:
        header = {"method": "LZW"}
        f.write((str(header) + "\n").encode())
        for code in compressed:
            f.write(code.to_bytes(2, byteorder='big'))

def compress_file(input_path, output_path):
    with open(input_path, 'r', encoding='utf-8') as f:
        text = f.read()

    compressed = lzw_compress(text)
    write_compressed_file(output_path, compressed)
    print(f"'{input_path}' dosyası LZW ile sıkıştırıldı -> '{output_path}'")
```

decompressor_lzw.py

```
# decompressor_lzw.py
import ast
```

```
def lzw_decompress(compressed):
    dict_size = 256
    dictionary = {i: chr(i) for i in range(dict_size)}
    result = []

    w = chr(compressed.pop(0))
    result.append(w)

    for k in compressed:
        if k in dictionary:
            entry = dictionary[k]
        elif k == dict_size:
            entry = w + w[0]
        else:
            raise ValueError("LZW hatası: sözlükte karşılık bulunamadı.")

        result.append(entry)
        dictionary[dict_size] = w + entry[0]
        dict_size += 1
        w = entry

    return ''.join(result)

def decompress_file(input_path, output_path):
    with open(input_path, 'rb') as f:
        header_line = f.readline()
        header = ast.literal_eval(header_line.decode().strip())
        method = header.get("method", "LZW")
        byte_data = f.read()

    compressed = [int.from_bytes(byte_data[i:i+2], byteorder='big') for i in range(0,
    text = lzw_decompress(compressed)

    with open(output_path, 'w', encoding='utf-8') as out:
        out.write(text)

    print(f"'{input_path}' dosyası açıldı ve '{output_path}' olarak kaydedildi.")
```


compressor_lossy.py

```
import string

def compress_file(input_path, output_path):
    with open(input_path, 'r', encoding='utf-8') as f:
        text = f.read()

        vowels = "aeiouAEIOU"
        punctuation = string.punctuation
        filtered = ''.join(c for c in text.lower() if c not in vowels and c not in punctuation)

        with open(output_path, 'w', encoding='utf-8') as f:
            f.write(str({"method": "Lossy"}) + "\n")
            f.write(filtered)

    return output_path
```

decompressor_lossy.py

```
def decompress_file(input_path, output_path):
    with open(input_path, 'r', encoding='utf-8') as f:
        lines = f.readlines()

        header = lines[0]
        compressed = ''.join(lines[1:])

        message = "[Kayıplı sıkıştırma: özgün veri geri getirilemez]\n" + compressed

        with open(output_path, 'w', encoding='utf-8') as f:
            f.write(message)

    return output_path
```

References

- [1] C. E. Shannon,
A Mathematical Theory of Communication,
- [2] D. A. Huffman,
A Method for the Construction of Minimum-Redundancy Codes,
- [3] T. A. Welch,
A Technique for High-Performance Data Compression,

- [4] R. M. Fano,
The Transmission of Information,
MIT Research Laboratory of Electronics, 1949.
- [5] T. M. Cover, J. A. Thomas,
Elements of Information Theory,
Wiley-Interscience, 2nd Edition, 2006.
- [6] S. Arora, B. Barak,
Computational Complexity: A Modern Approach,
Cambridge University Press, 2009.
- [7] S. Arora,
Computational Complexity - Lecture Notes,
Princeton University, 2007.
- [8] Python Software Foundation,
Tkinter Documentation,
<https://docs.python.org/3/library/tkinter.html>
- [9] Guido van Rossum et al.,
The Python Language Reference,
<https://www.python.org/doc/>
- [10] Internet Engineering Task Force (IETF),
RFC Archive,
<https://www.rfc-editor.org/>